DDR PHY

Complete Learning Guide

For Memory Controller Designers

What this document covers

A structured, bottom-up guide to understanding the DDR Physical Layer (PHY) — from first principles through every training algorithm. Organised into five blocks that build on each other, with diagrams at every stage. Designed specifically for someone building a memory controller who needs to understand what the PHY is doing and why.

Five Blocks

Block 1 — Foundations: PHY, signal integrity, differential & source synchronous signalling, skew model

Block 2 — Clock Generation: PLL, DLL, programmable delay lines

Block 3 — PHY Architecture: TX path, RX path, ZQ calibration, DFI

Block 4 — Training Algorithms: all six training steps in boot order

Block 5 — Runtime: PVT drift, periodic recalibration, low power

Date: March 2026

Contents

BLOCK 1
Foundations

1 What is a PHY and Why Does It Exist?

1.1 The Core Abstraction

A DDR PHY (Physical Layer) is the circuit block that sits between your digital Memory Controller (MC) and the actual DRAM devices on the board. Its job is to translate the digital world of your controller into the high-speed analog signals the DRAM expects, and vice versa.

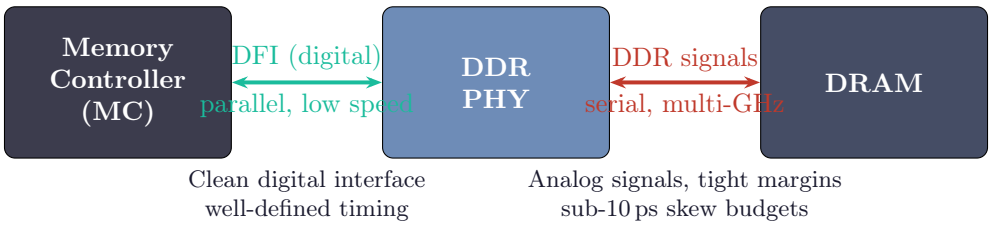

Figure 1: The PHY as abstraction boundary between digital controller and analog DRAM interface.

Key Point: Why the PHY exists

At DDR5-6400 speeds, the signals on the PCB are fundamentally analog problems — signal integrity, impedance matching, noise, jitter. Your memory controller should not need to know about any of that. The PHY provides a clean digital interface (DFI) to the controller while handling all the analog complexity on the DRAM side. This is the same reason you have a SerDes in an Ethernet PHY or a USB PHY — high-speed serial interfaces always need a dedicated physical layer block.

1.2 What the PHY Actually Does

On the **transmit (write) path**:

- Receives parallel data from the MC over the DFI bus
- Serialises it into a high-speed DQ stream (2 bits per DQ per clock cycle for DDR)
- Generates DQS (Data Strobe) aligned to the data
- Drives output buffers with controlled impedance onto the PCB traces

On the **receive (read) path**:

- Uses incoming DQS to clock a capture flip-flop for each DQ bit
- Deserialises the captured data back into parallel words
- Pushes the data into a FIFO to cross back into the controller clock domain
- Signals the MC with a valid flag when data is ready

Beyond data movement, the PHY also runs the full **training and calibration** sequence at boot to compensate for all the physical imperfections of the board and silicon.

2 Signal Integrity Basics

2.1 Why Signals Degrade at High Speed

At low frequencies you can treat PCB traces as simple wires. At multi-GHz speeds they behave as **transmission lines**, and you must account for:

- **Reflections** — if the trace impedance mismatches the driver or receiver impedance, part of the signal bounces back and corrupts the waveform
- **Crosstalk** — signals on adjacent traces couple electromagnetically and inject noise into each other
- **Skin effect** — at high frequencies current flows only on the outer surface of conductors, increasing resistance and causing losses
- **Dielectric loss** — the PCB material itself absorbs energy at high frequencies

Characteristic Impedance of a PCB Trace

$$Z_0 = \frac{1}{2\pi}\sqrt{\frac{L}{C}} \approx \frac{87}{\sqrt{\varepsilon_r + 1.41}} \ln\!\left(\frac{5.98\,h}{0.8\,w + t}\right)\ \Omega$$

where ε_r is the dielectric constant of the PCB material, h is the height above the ground plane, w is the trace width, and t is the trace thickness. DDR4/5 targets $Z_0 = 40$–$50\,\Omega$ for DQ traces.

2.2 Termination

To prevent reflections, the trace must be terminated at its characteristic impedance. DDR4 uses **On-Die Termination (ODT)** — resistors built inside the DRAM die that switch on only when needed. This is far better than external resistors because:

- No extra board components
- Can be enabled/disabled per transaction
- Calibrated via ZQ to track PVT variation (covered in Block 3)

3 Differential Signalling

3.1 Single-Ended vs. Differential

DDR uses a mix of both. The clock (CK/CK#) and data strobe (DQS/DQS#) are **differential pairs**. Data lines (DQ) are single-ended.

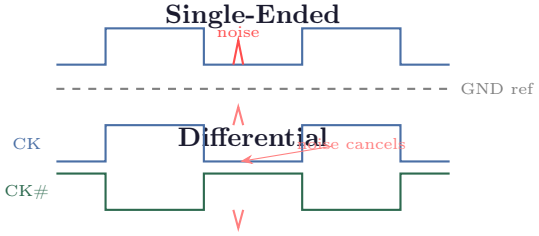

Figure 2: Differential signalling: noise couples equally onto both lines and cancels at the receiver, which only measures the difference $V_{CK} - V_{CK\#}$.

Why CK and DQS are differential

The receiver compares $V_{CK} - V_{CK\#}$ rather than V_{CK} vs a fixed reference. Any noise that couples onto the PCB affects both lines equally (common-mode noise), so it cancels out in the subtraction. This gives roughly 3× better noise immunity compared to single-ended, which is critical for the clock signals where jitter directly eats into the timing budget.

4 Source Synchronous Signalling

4.1 The Core Idea

In a traditional synchronous system, a global clock at the receiver is used to sample all incoming data. This breaks at multi-GHz speeds because the data arrives at slightly different times on every line, and the global clock has a fixed phase that may not align with any of them.

Source synchronous solves this by sending the strobe *alongside* the data from the same transmitter. Whatever delay the data experiences through PCB traces, connectors, and package, the strobe experiences roughly the same delay — they drift together.

Figure 3: Source synchronous: DQS is launched with DQ from the same transmitter and travels the same path. The receiver uses DQS edges to sample DQ, so absolute delay doesn't matter — only the relative timing between DQS and DQ matters.

Why training still exists

Source synchronous is a great idea in theory. In practice, DQS and DQ don't travel on perfectly identical paths — they have different trace lengths on the PCB (routing constraints), different package parasitics, different process variation on the drivers. Training measures these residual differences and programs delay compensation into the PHY's delay lines to correct them. That's the entire motivation for everything in Block 4.

5 The Inter-Lane Skew Problem: Why Training Exists

5.1 The Delay Model

Every signal from the PHY to the DRAM travels through a path with total delay:

$$\tau_i = t' + \delta_i$$

- t' = nominal propagation delay, common to all lanes (package + die + average PCB trace)
- δ_i = per-lane deviation from nominal, can be positive or negative

- i indexes the lane: $i \in \{0, 1, \ldots, N_{lanes} - 1\}$

The **inter-lane skew** is: $W_\delta = \delta_{max} - \delta_{min}$

5.2 Sources of δ_i

Source	Mechanism	Typical $\lvert\delta_i\rvert$
PCB trace length mismatch	Different routing lengths due to board constraints	10–150 ps
PCB dielectric variation	Non-uniform ε_r changes propagation velocity	2–15 ps
Package pin inductance	Different bond-wire inductance per pin	3–20 ps
On-die clock tree skew	H-tree imbalance reaching different TX flip-flops	2–10 ps
Thermal gradient	Velocity varies with temperature $\partial\tau/\partial T \approx 0.5\,\text{ps}/°\text{C}/\text{cm}$	1–10 ps
Process variation	MOSFET V_t and C_{ox} variation changes driver timing	2–15 ps

5.3 Why This Breaks Everything

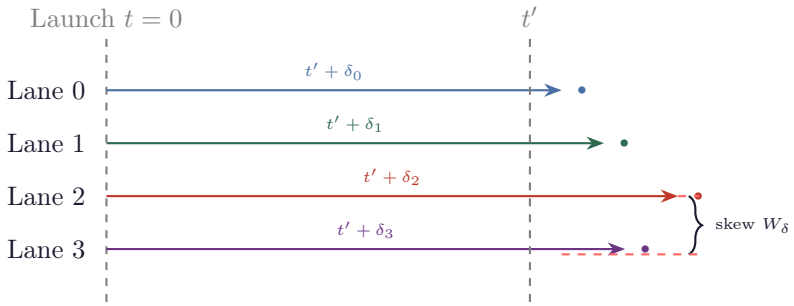

Figure 4: All lanes launch simultaneously at $t = 0$ but arrive at the DRAM at different times $t' + \delta_i$. The DRAM must sample all of them with its internal clock — impossible without compensation. At DDR5-6400 the total skew budget is under 8 ps.

The key insight: δ_i has three properties that force training

1. **Unknown at design time** — PCB routing is finalised after chip tape-out
2. **Unknown at power-on** — must be measured by the PHY itself during boot
3. **Slowly varying at runtime** — changes with temperature, requiring periodic re-calibration

This is why every DDR system must run training at boot and re-calibrate periodically. Training is not optional — it is the only way the system can function.

5.4 The Unit Interval and Why It Matters

$$UI = \frac{1}{2 \times f_{CK}}$$

DDR Rate	f_{CK}	UI	Allowed skew ($\leq 5\%$ UI)
DDR4-3200	1600 MHz	312 ps	15.6 ps
DDR5-4800	2400 MHz	208 ps	10.4 ps
DDR5-6400	3200 MHz	156 ps	7.8 ps

At DDR5-6400, **the entire skew budget across all sources is under 8 ps**.

BLOCK 2
Clock Generation

6 Phase Locked Loop (PLL)

6.1 What a PLL Does

A PLL takes a low-frequency reference clock (typically 100 MHz from a crystal oscillator on the board) and multiplies it up to the high frequency needed for DDR operation (e.g. 1600 MHz for DDR4-3200, 3200 MHz for DDR5-6400).

Core idea in one sentence

A PLL watches the difference between its output clock and a reference clock, and continuously corrects itself until they match — locking the output phase to the reference.

6.2 PLL Block Diagram

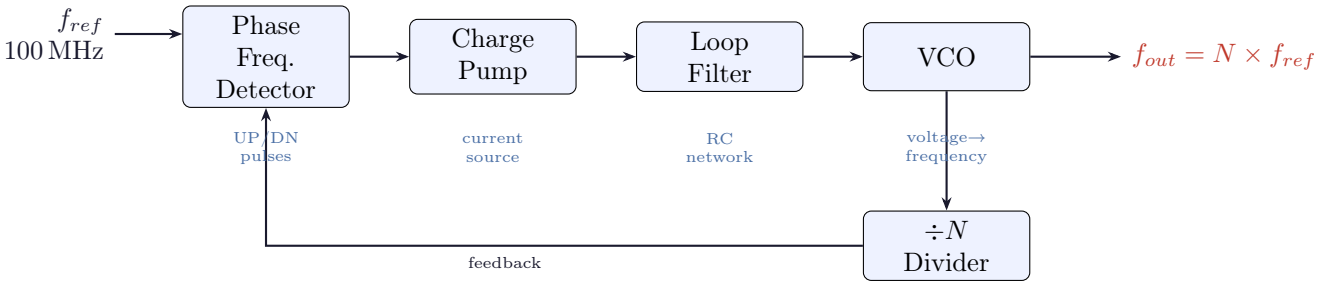

Figure 5: PLL block diagram. The PFD compares reference vs feedback, the charge pump converts the error to current, the loop filter smooths it to a DC voltage, and the VCO generates the output. The $\div N$ in feedback forces $f_{out} = N \times f_{ref}$.

6.3 The Three Main Blocks Explained

Phase Frequency Detector (PFD): Compares the reference clock against the divided feedback clock and produces UP or DOWN pulses based on which edge arrives first. UP means "output is too slow", DOWN means "output is too fast".

Charge Pump + Loop Filter: Converts UP/DOWN pulses into a smooth DC voltage. The loop filter (an RC network) determines the loop bandwidth — how quickly the PLL reacts. Too fast and it hunts around noisily; too slow and it can't track changes.

VCO (Voltage Controlled Oscillator): Generates the output clock. Higher input voltage → higher frequency. This is the fundamentally analog heart of the PLL — it's a ring oscillator or LC tank where a voltage physically changes the oscillation frequency by tuning varactor capacitances or transistor bias points.

Why PLL jitter matters so much

The VCO is a free-running oscillator — any noise on its control voltage becomes frequency modulation, which integrates into phase noise (jitter) on the output clock. This jitter feeds directly into every output driver through the clock distribution tree. At DDR5-6400 with a

7.8 ps skew budget, even 1–2 ps of PLL jitter is significant. This is also why power supply noise (which couples into the VCO control voltage) is such a headache in DDR PHY design.

7 Delay Locked Loop (DLL)

7.1 DLL vs PLL

A DLL does *not* generate a new clock frequency. It takes an existing clock and adjusts its phase by passing it through a chain of delay elements, locking the total delay to exactly one clock period.

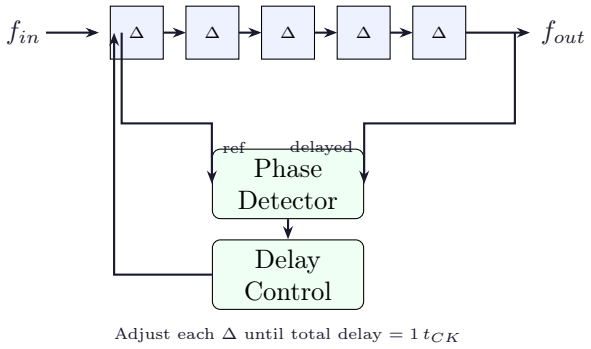

Figure 6: DLL block diagram. The delay chain introduces a controlled delay; the phase detector compares the input to the delayed output and adjusts all delay cells until total delay equals exactly one clock period.

Why DLL doesn't accumulate jitter

Unlike a PLL with a free-running VCO, a DLL just passes the input clock through delay cells. Input jitter passes through unchanged — but it doesn't *accumulate*. A PLL can turn supply noise into growing phase error over time. A DLL's phase error is always bounded by the input jitter plus the delay cell noise. For fine phase alignment inside the PHY this property is very valuable.

7.2 Why DDR PHY Uses Both

Block	Purpose	Why this one?
PLL	Frequency multiplication ($\times N$)	Only PLL can generate a new frequency
DLL	Phase alignment & deskew	Doesn't accumulate jitter; simpler stability

In a typical DDR PHY: the PLL takes the 100 MHz reference and generates the 1600–3200 MHz high-speed clock. The DLL then takes that clock and deskews it across the PHY's internal clock distribution tree so every output driver sees the clock at exactly the right phase.

8 Programmable Delay Line (DCDL)

8.1 The Mechanism Behind Every Training Algorithm

Every training algorithm in Block 4 ultimately programs a **delay code** into a Digitally Controlled Delay Line (DCDL). Understanding this structure is essential before tackling any training algorithm.

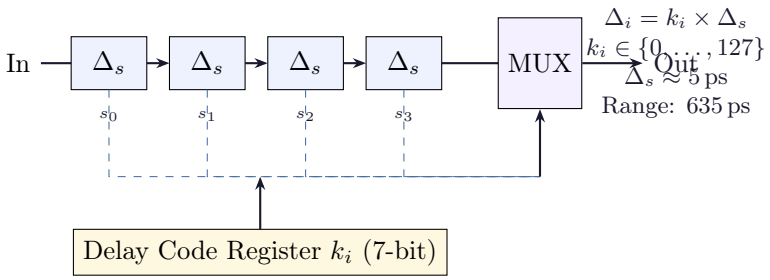

Figure 7: DCDL architecture. A chain of identical delay cells feeds a MUX. The delay code register selects which tap to use as output. Each code step is $\Delta_s \approx 5\,\text{ps}$; a 7-bit code gives 128 steps covering 0–635 ps of range.

The unified view of all training algorithms

Every training algorithm from Write Levelling to CA Training is doing the same thing at its core: finding the right value of k_i to program into the DCDL for a particular signal path. The algorithms differ only in *which* delay line they're calibrating and *what feedback mechanism* they use to measure pass/fail.

BLOCK 3
PHY Architecture

9 PHY TX Path

9.1 From DFI Data to DDR Signals

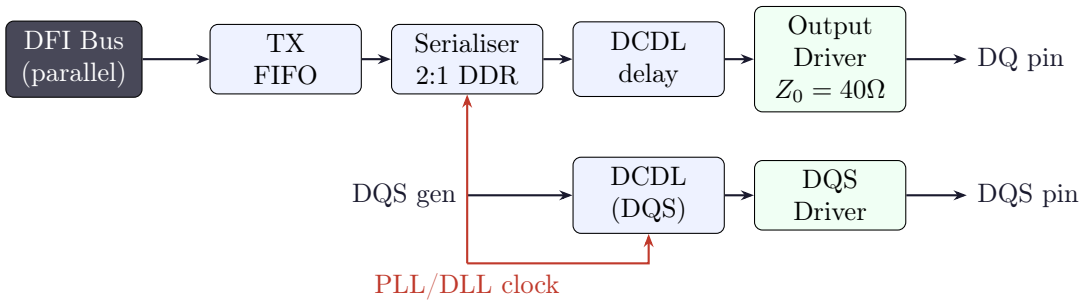

Figure 8: PHY TX path per byte lane. DFI parallel data enters a FIFO, gets serialised 2:1 for DDR, passes through a DCDL for timing adjustment, then drives the output buffer onto the PCB. DQS is generated separately and also passes through its own DCDL.

9.2 Output Drivers in a DDR PHY

The output drivers are every circuit that drives a signal onto the physical PCB:

- **DQ drivers** — one per DQ pin, 8 per byte lane (72 total for ×72 channel)
- **DQS/DQS# drivers** — differential strobe pair, one per byte lane
- **DM/DBI drivers** — data mask or bus inversion, one per byte lane
- **CA drivers** — A[0]–A[16], BA[1:0], BG[1:0], ∼20 signals
- **CK/CK# drivers** — differential clock, most timing-critical driver in the PHY
- **CS# drivers** — one per rank
- **CKE drivers** — one per rank
- **ODT drivers** — on-die termination control, one per rank

For a DDR4 ×72 channel with 2 ranks: easily **over 150 output drivers**, all requiring the clean low-jitter clock from the PLL/DLL.

10 PHY RX Path

10.1 Capturing Read Data

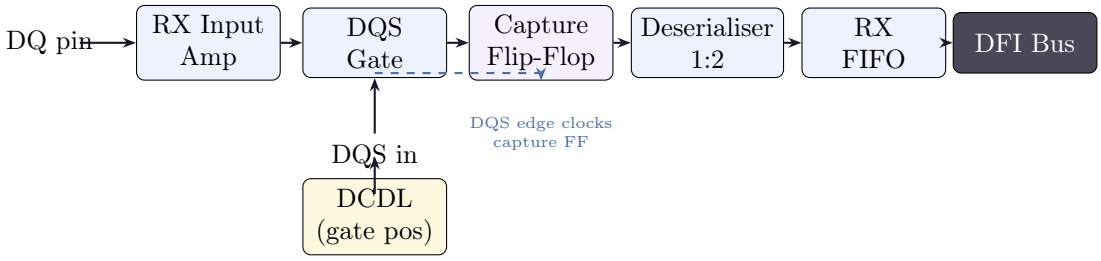

Figure 9: PHY RX path per byte lane. The DQS gate opens only during valid bursts. Each DQ bit is captured by a flip-flop clocked by DQS. Data is then deserialised and pushed into a FIFO to cross into the controller clock domain.

The DQS gate is why Read Gate Training exists

Before the read burst begins, DQS is floating (Hi-Z). If the gate is open during this time, the capture flip-flop sees noise and latches garbage. The gate must open *exactly* at the DQS preamble — not before (captures noise), not after (misses first bits). Since the preamble arrival time depends on the unknown $\delta_{DQS,i}$, it must be measured and programmed via training.

11 ZQ Impedance Calibration

11.1 Why Impedance Needs Calibration

The output driver impedance R_{ON} and ODT impedance R_{TT} are implemented as arrays of PMOS/NMOS transistors. Their resistance depends on:

$$R_{eff} = f(V_t,\, V_{DS},\, T,\, V_{DD})$$

All four quantities change with PVT (Process, Voltage, Temperature). Without calibration R_{ON} can vary by $\pm 30\%$ — enough to cause severe reflections.

11.2 The SAR Algorithm

The PHY has an external precision resistor $R_{ZQ} = 240\,\Omega$ on the ZQ pin. A Successive Approximation Register (SAR) loop binary-searches for the driver code that matches this reference:

ZQ SAR Calibration — Step by Step

1. Set pull-up code $C_{PU} = 0$ (all legs off)
2. For bit $b = B - 1$ down to 0 (B = 8 bits, binary search):
   - Set bit b of C_{PU} to 1
   - Wait $t_{settle} \approx 20\,\text{ns}$
   - If $V_{ZQ} < V_{DD}/2$: R_{ON} too high $\Rightarrow$ keep bit set
   - If $V_{ZQ} > V_{DD}/2$: R_{ON} too low $\Rightarrow$ clear bit
3. After 8 steps: C^*_{PU} locked. Accuracy: $\Delta R_{ON}/R_{ON} = 1/256 \approx 0.4\%$

4. Mirror C^*_{PU} to pull-down with NMOS/PMOS mobility correction: $C^*_{PD} = C^*_{PU} \times (\mu_p/\mu_n) \approx C^*_{PU} \times 0.5$

Periodic recalibration (ZQCS) runs every 128 ms because $\partial R_{ON}/\partial T \approx +0.3\%/°\text{C}$ and a 50°C swing causes ∼15% drift.

12 DFI Interface

The DFI (DDR PHY Interface) is the standard digital interface between your Memory Controller and the PHY. It defines seven signal groups:

Group	Direction	Purpose
Control	MC→PHY	Address, bank, command signals (d_address, d_cs_n, etc.)
Write Data	MC→PHY	Data payload, enables, byte masks (d_wrdata, d_wrdata_en)
Read Data	Bidirectional	Read enable (MC→PHY); data + valid (PHY→MC)
Update	Bidirectional	Handshake for atomic config updates (ctrlupd, phyupd)
Status	Mixed	Init trigger, freq ratio, byte disable, parity
Training	Bidirectional	Per-lane read/write levelling control and delay codes
Low Power	Bidirectional	PHY power-down request/acknowledge, wakeup budget

The key parameters your controller must know: `tphy_wrlat` (write latency in DFI clocks), `tphy_rdlat` (read latency), and `dfi_freq_ratio` which scales all timing parameters by the DFI:DRAM clock ratio.

BLOCK 4
Training Algorithms

Boot execution order (JEDEC mandated)

ZQ Long → Write Levelling → Read Gate Training → Read Centering → Write Centering → CA Training → 2D VREF + Timing Scan → DFE Training (DDR5 only)

Each step depends on the previous step having succeeded. The order cannot be changed.

13 Write Levelling

13.1 What It Corrects

Write levelling corrects the **DQS-to-CK phase offset** at each DRAM caused by the fly-by topology. On a real DIMM, the CK signal daisy-chains across DRAMs rather than arriving simultaneously (star topology would cost too much PCB area).

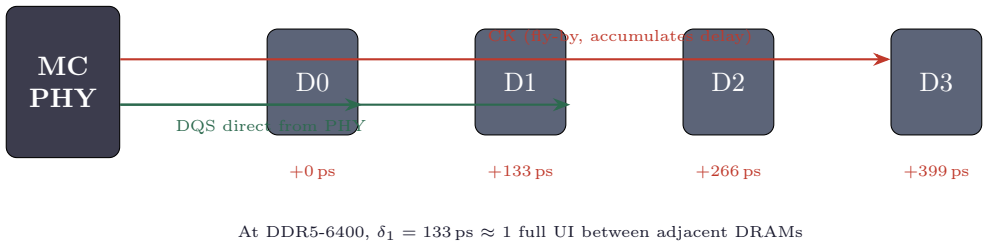

Figure 10: Fly-by topology: CK accumulates delay across DRAMs but DQS comes directly from the PHY. Each DRAM sees a different CK phase relative to DQS. Write levelling measures and corrects this.

13.2 Algorithm

Write Levelling — Step by Step

1. Issue MRS to enable Write Levelling Mode (MR1[A7]=1). DRAM now samples CK on every DQS rising edge and drives DQ[0] = sampled CK value.
2. Set DQS TX delay code $k = 0$.
3. Sweep k from 0 to $K_{max} = 127$:
   - Apply code k to DQS TX DCDL
   - Pulse DQS once (single rising edge)
   - Read DQ[0] feedback from DRAM
   - If DQ[0] transitions $0 \rightarrow 1$: record k_{trans}, exit loop
4. Refine: sweep ±8 steps around k_{trans}, find stable window W_{stable}
5. $k_i^* = k_{trans} + \lfloor W_{stable}/2 \rfloor$
6. Disable Write Levelling Mode. Repeat for all lanes.

Result: $\Delta_{WL,i}$ compensates δ_i on the write DQS path. Residual error $\leq \Delta_s/2 \approx 2.5\,\text{ps}$.

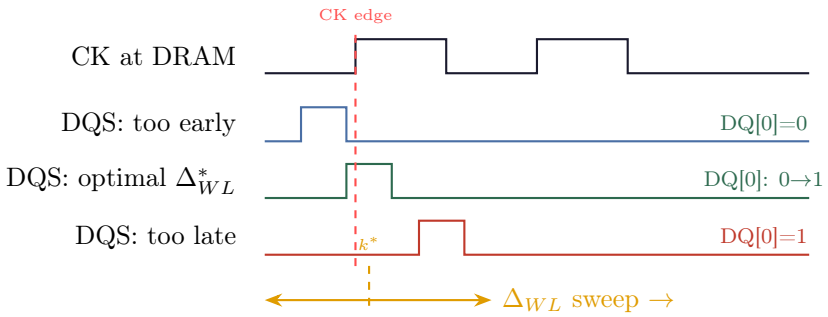

Figure 11: Write levelling waveform. PHY sweeps DQS delay. DQ[0]=0 when DQS is early (before CK edge); DQ[0]=1 when DQS is late (after CK edge). The 0→1 transition marks optimal Δ^*_{WL}.

14 Read DQS Gate Training

14.1 What It Corrects

During a READ, the DRAM drives DQS only for the burst duration. Before the preamble, DQS is Hi-Z (floating). The PHY must open a gate *exactly* at the preamble — not before (captures noise), not after (misses data).

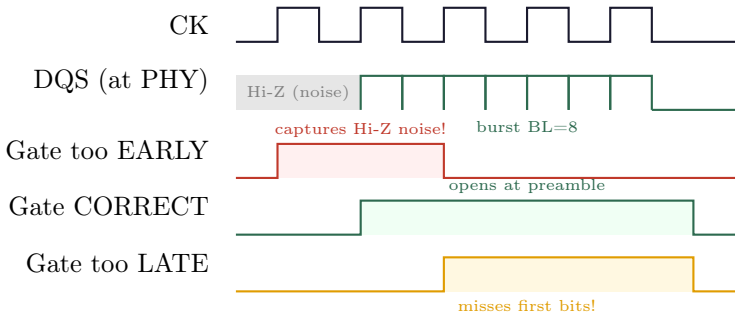

Figure 12: Read DQS Gate Training. Three gate positions shown. Only the correct position opens exactly at the DQS preamble, capturing the full burst without noise.

Read Gate Training Algorithm

1. Issue READ command with a known pattern (0xAA55 per byte)
2. Sweep gate open-position code k from 0 to K_{max}
3. For each k: capture returned data, compare to expected, record PASS/FAIL
4. Find passing window $[k_L, k_R]$
5. Set optimal gate position: $k^*_{gate,i} = \lfloor (k_L + k_R)/2 \rfloor$
6. Repeat for each lane

Must complete before Read Centering — the gate must be open correctly before

per-bit timing can be measured.

15 Read DQ/DQS Centering

15.1 What It Corrects

After gate training, DQS is approximately aligned. Now each DQ bit's RX sampling point must be placed at the *centre* of its valid window (eye). This is the per-bit, fine-grain step.

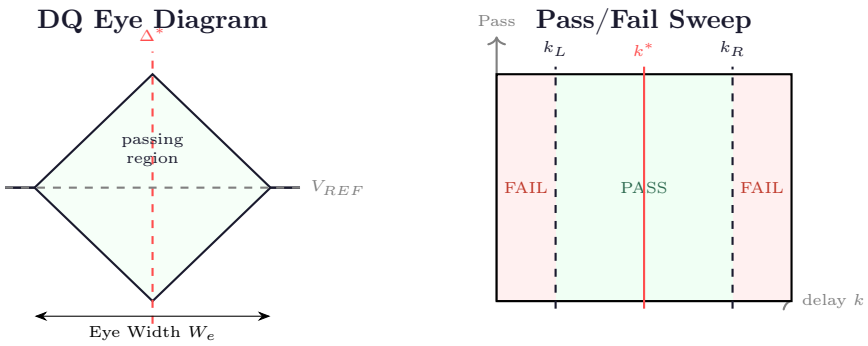

Figure 13: Left: DQ bit eye diagram. The optimal sampling point is the geometric centre of the eye. Right: pass/fail result of the delay sweep. PHY finds k_L and k_R then sets $k^* = (k_L + k_R)/2$.

Read DQ Centering Algorithm (per bit, per lane)

1. Transmit known repeating pattern (PRBS7 or 0xAA/0x55)
2. For each delay code k from 0 to K_{max}:
   - Set RX Phase Interpolator code to k for bit b of lane i
   - Capture $N_{samples} = 128$ read bursts; record PASS if 0 errors
3. Find: $k_L = \min\{k : \text{Pass}(k) = 1\}$, $k_R = \max\{k : \text{Pass}(k) = 1\}$
4. Optimal code: $k^*_{i,b} = \lfloor (k_L + k_R)/2 \rfloor$
5. Eye width: $W_{i,b} = (k_R - k_L) \times \Delta_s$ [ps]
6. Flag FAIL if $W_{i,b} < 0.2\, t_{CK}$

16 Write DQ/DQS Centering

16.1 What It Corrects

After write levelling aligns DQS to CK, the DQ bits must be centred within the DQS window at the DRAM input. Both DQS and DQ are driven by the PHY; their relative timing is set by independent TX delay lines.

Write Data Setup and Hold Requirements at DRAM input:

$$t_{DS,i,b} = t_{DQS\ edge} - t_{DQ\ valid,i,b} \geq t_{DS,min}$$

$$t_{DH,i,b} = t_{DQ\ valid,i,b,hold} - t_{DQS\ edge} \geq t_{DH,min}$$

Both satisfied when DQ is launched $\frac{1}{4}$ cycle before the DQS edge:

$$\Delta^{TX}_{DQ,i,b} = \Delta^{TX}_{DQS,i} - \frac{t_{CK}}{4}$$

Write DQ Centering Algorithm (write-then-read loop)

1. Select test pattern P (PRBS15)
2. Fix DQS TX delay at $\Delta^*_{WL,i}$ (from write levelling)
3. For TX DQ delay code k from 0 to K_{max}:
   - Write pattern P using DQ delay code k
   - Read back the same address
   - Compare read data against P; record PASS/FAIL
4. $k^* = (k_L + k_R)/2$; write eye width $= (k_R - k_L) \times \Delta_s$

Why write-then-read: The DRAM has no feedback mechanism during writes. The PHY writes a pattern and reads it back using the already-trained read path. This is why write centering *must follow* read gate training and read centering.

17 2D VREF + Timing Scan

17.1 Why 1D Is Not Enough

A 1D timing scan finds the horizontal eye centre. But the optimal sampling point also depends on the threshold voltage V_{REF}. The true optimum in the (time, voltage) plane may be offset from both centres due to eye asymmetry.

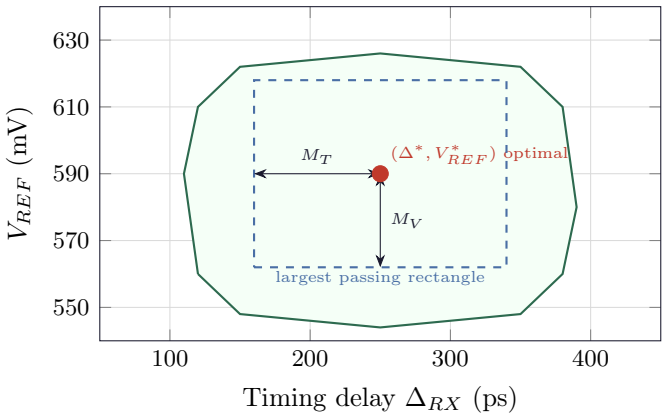

Figure 14: 2D Eye Scan. Each grid point is a (VREF, timing) pair tested by writes and reads. The largest error-free rectangle defines the optimal operating point (Δ^*, V^*_{REF}) with timing margin M_T and voltage margin M_V.

2D VREF + Timing Algorithm

1. Outer loop: sweep V_{REF} from V_{min} to V_{max} in steps of ΔV
   - Programme VREF (DDR4: Mode Register 6; DDR5: on-die trim code)
   - Wait $t_{VREF\ settle} \geq 150\,\text{ns}$
   - Inner loop: sweep timing code k from 0 to K_{max}
   - Issue write + read-back; record pass/fail in matrix $M[v][k]$
2. Find largest contiguous 2D passing rectangle in M
3. Set $V_{REF} = v^*$, RX delay = k^* at centre of rectangle

Total test points: $\sim 50 \times 128 = 6400$ per bit per lane.
Minimum acceptable: $M_T \geq 0.1 \times t_{CK}$, $M_V \geq 30\,\text{mV}$.

18 CA Training

18.1 What It Corrects

The Command/Address bus must arrive at the DRAM with correct setup/hold timing relative to CK. PCB trace differences cause a CA-to-CK skew $\delta_{CA,b}$ that must be measured and compensated.

Unlike DQ (source-synchronous with DQS), CA signals are registered against the DRAM's local CK, so they cannot use DQS as a reference. CA training uses a special DRAM feedback mode instead.

CA Training Algorithm

1. Enter CA Training Mode via MRS. DRAM echoes captured CA value via DQ[0]: DQ[0] = $f(\text{CA}[n], \text{CK edge})$
2. Sweep CA launch delay Δ_{CA} from $-N$ to $+N$ steps
3. For each Δ_{CA}: issue CA pulse, read DQ[0] feedback
   - DQ[0]=0: CA sampled outside valid window
   - DQ[0]=1: CA correctly sampled
4. Find passing window $[k_L, k_R]$; optimal: $k^*_{CA} = (k_L + k_R)/2$
5. Apply k^*_{CA} to CA TX delay line

19 DFE Tap Training (DDR5)

19.1 Why DFE Is Needed

At DDR5-6400 (6.4 Gbps), the PCB channel has significant ISI (Inter-Symbol Interference) — distortion where previous bits bleed into the current bit. A typical DDR5 channel has 5–10 dB loss at the 3.2 GHz Nyquist frequency.

ISI Model:

$$r[k] = h_0 \cdot d[k] + \underbrace{\sum_{j=1}^{N} h_j \cdot d[k-j]}_{\text{ISI from past } N \text{ bits}} + n[k]$$

The DFE (Decision Feedback Equaliser) subtracts estimated ISI using previously decided bits $\hat{d}[k-j]$.

DFE LMS Training Algorithm

1. Transmit known training sequence (PRBS31)
2. DFE output: $y[k] = r[k] - \sum_{j=1}^{N} c_j \hat{d}[k-j]$
3. Slicer decision: $\hat{d}[k] = \text{sign}(y[k])$
4. Error: $e[k] = \hat{d}[k] - y[k]$
5. LMS update: $c_j[k+1] = c_j[k] + \mu \cdot e[k] \cdot \hat{d}[k-j]$
6. Iterate until $|e[k]| < \varepsilon$

Typical: $N = 4$ taps, $\mu = 2^{-8}$, converges in $\sim$1000 symbols. **DFE must train last** — tap coefficients are only stable once the slicer timing is correct.

BLOCK 5
Runtime Behaviour

20 Why Boot-Time Training Is Not Enough: PVT Drift

PVT Drift During Operation

After training at $T_0 = 25°C$, the system heats to $T_1 = 75°C$ under load. Delay change on a 20 mm PCB trace:

$$\Delta\tau_{PCB} \approx \frac{\partial t_{pd}}{\partial T} \times \Delta T \approx 0.015\,\text{ps/mm/°C} \times 20\,\text{mm} \times 50°C = 15\,\text{ps}$$

At DDR5-6400 (UI = 156 ps), a 15 ps shift eats ≈10% of UI in timing margin. Without re-calibration the system approaches the timing failure boundary.

Similarly for impedance: $\partial R_{ON}/\partial T \approx +0.3\%/°C$. At 50°C swing: $\Delta R_{ON} \approx 15\%$ — enough to significantly degrade signal integrity if not recalibrated.

21 Periodic Recalibration Schedule

Calibration	Trigger	Frequency
ZQCS (impedance)	Periodic scheduler; temp sensor alert	Every 128 ms
DLL re-lock	Clock frequency change, VDD change	On-demand
VREF adjustment	Temperature $\Delta T > 10°C$	Periodic
Duty Cycle Correction	PLL phase monitor; CK duty $< 47\%$	On-demand
Partial re-training	Self-refresh exit; mode register change	On-demand
Full re-training	Cold boot; warm reset; frequency change	Boot only

Table 1: Runtime re-calibration events and triggers.

22 Online Margin Monitoring

Modern DDR5 PHYs implement **online eye monitoring** — continuously measuring timing margin in the background without interrupting normal operation:

$$\hat{M}_T(t) = \hat{k}_R(t) - \hat{k}_L(t) \quad \text{(measured continuously)}$$

When $\hat{M}_T(t) < M_{threshold}$, a partial retrain is triggered automatically without a full system reset. The threshold is typically set to leave enough margin that even if recalibration takes a few microseconds, the system hasn't yet hit an actual error.

23 Low Power States and PHY Wakeup

When the memory bus is idle, the PHY can power down internal blocks to save energy. The DFI Low Power interface coordinates this:

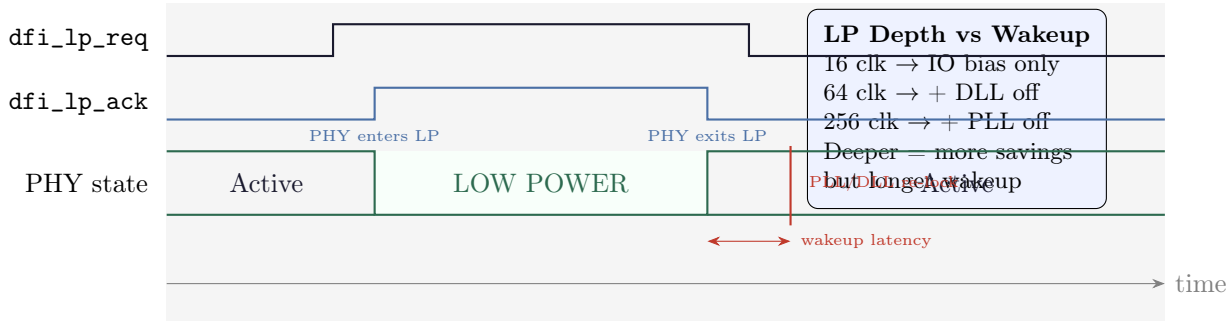

Figure 15: PHY low power handshake. MC asserts `lp_req`; PHY powers down and asserts `lp_ack`. On wakeup, MC deasserts `lp_req` and waits for PHY to deassert `lp_ack` before issuing any command. Wakeup latency depends on LP depth — deeper sleep = longer wakeup.

The dfi_lp_wakeup signal

The MC sends `dfi_lp_wakeup` *before* asserting `dfi_lp_req`. It encodes the expected idle duration (4-bit code: 16 clocks to 256+ clocks) so the PHY can decide how deeply to power down. Longer idle → PHY shuts down more blocks (DLL, PLL, IO) → bigger power saving but longer wakeup time. The MC must never issue a transaction until it sees `dfi_lp_ack` deasserted.

Summary: The Complete Mental Model

If you remember one thing from each block:

Block 1 (Foundations): The PHY exists because multi-GHz DDR signals are fundamentally analog problems. Source synchronous signalling sends the strobe with the data, but imperfect routing creates residual skew δ_i that forces training.

Block 2 (Clocks): The PLL multiplies frequency (accumulates jitter). The DLL aligns phase (doesn't accumulate jitter). The DCDL is what training actually programs — every algorithm finds the right delay code k_i for a specific signal path.

Block 3 (Architecture): TX path serialises DFI data to DDR stream via output drivers. RX path uses DQS-gated capture flip-flops to deserialise. ZQ calibrates impedance. DFI is the clean digital boundary to your controller.

Block 4 (Training): All algorithms answer the same question in different contexts: *what delay code puts signal X in the right phase relationship to signal Y?* ZQ is the exception — it corrects impedance, not delay.

Block 5 (Runtime): Temperature causes $\sim$0.015 ps/mm/°C delay drift and $\sim$0.3%/°C impedance drift. ZQCS every 128 ms and periodic partial retraining keep the system inside its margins. Low power handshake ensures the PHY is fully awake before any transaction.

Training Order Quick Reference

#	Algorithm	Corrects	Time
1	ZQ Long Calibration	R_{ON}, R_{TT} impedance	$\sim$1 μs
2	Write Levelling	DQS-to-CK skew (fly-by)	$\sim$5 μs
3	Read Gate Training	DQS gate open position	$\sim$3 μs
4	Read DQ/DQS Centering	Per-bit RX delay	$\sim$10 μs
5	Write DQ/DQS Centering	Per-bit TX delay	$\sim$10 μs
6	CA Training	CA-to-CK timing	$\sim$5 μs
7	2D VREF + Timing Scan	Optimal (V, t) operating point	$\sim$50 μs
8	DFE Tap Training	ISI (DDR5 only)	$\sim$20 μs
	Total boot training		**100–500 μs**